

SBNZ – Predlog projekta

**Sistem baziran na znanju za procenu isplativosti uvoza, formiranje
cena i upravljanje zalihama kozmetičkih proizvoda**

Članovi tima – Teodora Belić SV55/2022

Motivacija

Uvoz i distribucija kozmetičkih proizvoda predstavljaju složen proces koji uključuje veliki broj faktora (troškovi, konkurencija, potražnja i upravljanje zalihama). U praksi se ove odluke često donose na osnovu iskustva i subjektivne procene, uz korišćenje velikog broja Excel tabela, što može dovesti do nepreciznih i nekonzistentnih rezultata.

Dodatni problem predstavlja to što su odluke o uvozu, formiranju cena i upravljanju zalihama međusobno povezane, pa promena u jednom delu utiče na ostale, što dodatno otežava donošenje optimalnih odluka.

Motivacija ovog projekta je razvoj sistema koji povezuje ove procese i omogućava donošenje tačnijih i doslednijih odluka na osnovu znanja stručnjaka.

Pregled problema

U praksi, kompanije koje se bave uvozom i distribucijom kozmetike, kao što je Magic Beauty, koriste Excel tabele i interne analize. Ovakav pristup zahteva ručnu obradu podataka i oslanja se na iskustvo zaposlenih.

Postoje i napredniji sistemi (ERP i alati za upravljanje zalihama i cenama), ali oni:

- ne koriste formalizovano ekspertno znanje
- ne omogućavaju automatsko zaključivanje
- ne povezuju odluke o uvozu, ceni i zalihama
- ne pružaju objašnjenje odluka

Zbog toga postoji potreba za sistemom koji će objediniti ove procese i omogućiti donošenje odluka na osnovu jasno definisanih pravila.

Predloženi sistem se razlikuje po tome što:

- koristi ekspertna pravila
- omogućava forward i backward chaining
- koristi CEP za analizu prodaje i zaliha kroz vreme
- integriše više poslovnih procesa u jedinstven sistem

Prednost sistema je automatizovano, konzistentno i transparentno donošenje odluka. Konkretno, Magic Beauty trenutno ove poslove radi manuelno.

Metodologija rada

Ulazi u sistem

- nabavna cena proizvoda
- troškovi transporta (5-7%)
- carina (samo van EU)
- kategorija proizvoda (luxury, mass, professional)
- opseg konkurentskih cena (min–max)
- procena potražnje (0–10)
- marža (B2B – 30%; D2C – 45%)
- stanje zaliha
- istorija prodaje
- vreme isporuke

Izlazi iz sistema

- preporuka za uvoz (da/ne)
 - preporučena cena (B2B i D2C)
 - procena profita
 - procena rizika
 - preporuka za naručivanje zaliha
 - upozorenje (kritične zalihe)
 - objašnjenje odluke
-

Baza znanja i pravila sistema

Baza znanja organizovana je u grupe pravila koje pokrivaju različite aspekte odlučivanja u sistemu.

Procena ukupnih troškova

Sistem prvo računa ukupni trošak proizvoda. U ukupni trošak ulaze nabavna cena, troškovi transporta i carina.

Primer pravila:

- Ako su uneti nabavna cena, transport i carina, sistem računa ukupni trošak proizvoda.
- Ako je ukupni trošak nizak u odnosu na očekivanu prodajnu cenu, proizvod se označava kao potencijalno isplativ.
- Ako je ukupni trošak visok, sistem povećava procenu rizika.

Procena isplativosti uvoza

Na osnovu ukupnog troška, marži, potražnje i konkurentskog opsega cena, sistem donosi preporuku da li proizvod treba uvesti.

Primer pravila:

- Ako je potražnja visoka, troškovi niski i preporučena D2C cena se uklapa u konkurentski opseg → sistem preporučuje uvoz.
- Ako je preporučena D2C cena veća od maksimalne konkurentske cene → proizvod nije konkurentan → ne preporučiti uvoz.
- Ako je potražnja niska i očekivani profit mali → povećava se rizik i ne preporučuje uvoz.
- Ako su troškovi visoki i dovode do previsoke prodajne cene → proizvod nije isplativ za uvoz.

Formiranje B2B i D2C cena

Sistem koristi unapred definisane marže za formiranje cena:

- B2B marža = 30%
- D2C marža = 45%

Na osnovu ukupnog troška, sistem računa:

- $D2C \text{ cena} = \text{ukupni trošak} + 45\%$
- $B2B \text{ cena} = \text{ukupni trošak} + 30\%$

Primer pravila:

- Ako je poznat ukupni trošak proizvoda, sistem računa D2C cenu primenom marže od 45%.
 - Ako je poznat ukupni trošak proizvoda, sistem računa B2B cenu primenom marže od 30%.
 - Ako je preporučena cena iznad maksimalne konkurentske cene, sistem označava proizvod kao nekonkurentan.
-

Forward chaining rezonovanje

Sistem koristi višestepeno zaključivanje:

1. Računa se ukupni trošak
2. Procena troškova (niski / visoki)
3. Procena isplativosti
4. Donošenje odluke o uvozu
5. Formiranje cene
6. Procena prodaje
7. Upravljanje zaliha

Rezultat jednog pravila koristi se kao ulaz za naredno pravilo.

Accumulate funkcija

Accumulate funkcija omogućava agregaciju više podataka (npr. prodaja kroz više dana) kako bi se dobile nove vrednosti, poput prosečne prodaje ili ukupnog troška, koje se zatim koriste u pravilima sistema.

Sistem koristi agregaciju podataka:

- $\text{ukupni trošak} = \text{nabavna cena} + \text{transport} + \text{carina}$
- $\text{prosečna prodaja} = \text{ukupna prodaja} / \text{broj dana}$
- $\text{preostali dani zaliha} = \text{stanje zaliha} / \text{prosečna prodaja}$

Primer pravila:

- Ako je prosečna dnevna prodaja veća od 5 → visoka prodaja
 - Ako je prosečna dnevna prodaja manja od 2 → niska prodaja
 - Ako je preostalo manje od 7 dana zaliha → upozorenje
-

Upravljanje zalihama

Sistem prati stanje zaliha i prodaju.

Primer pravila:

- Ako je stanje zaliha manje od 10 → upozorenje
 - Ako je stanje zaliha manje od 5 → hitno upozorenje
 - Ako je prodaja visoka i zalihe niske → ranije naručivanje
 - Ako je prodaja niska i zalihe velike → smanjenje narudžbine
 - Ako je proizvod isplativ i prodaja visoka → povećanje količine
-

Complex Event Processing (CEP)

CEP se koristi za praćenje vremenskih obrazaca u prodaji i zalihama. Sistem prima događaje kao što su nova prodaja, promena stanja zaliha i prijem nove isporuke. Na osnovu više događaja u određenom vremenskom prozoru sistem otkriva trendove, ubrzano pražnjenje zaliha i rizik od nestašice proizvoda.

Primer događaja:

```
SaleEvent(productId, quantity, timestamp)
StockChangeEvent(productId, currentStock, timestamp)
DeliveryEvent(productId, deliveredQuantity, timestamp)
```

Primeri CEP pravila:

```
rule "Rast prodaje u tri uzastopna perioda"
when
    $p : Product()
    $s1 : SalesPeriod(productId == $p.id, period == 1, quantitySold > 0)
    $s2 : SalesPeriod(productId == $p.id, period == 2, quantitySold > $s1.quantitySold)
    $s3 : SalesPeriod(productId == $p.id, period == 3, quantitySold > $s2.quantitySold)
then
    insert(new GrowingSalesTrend($p.id));
end
```

```
rule "Brzo smanjenje zaliha"
when
    $p : Product()
    $trend : GrowingSalesTrend(productId == $p.id)
    $stock : Stock(productId == $p.id, quantity < reorderThreshold)
then
    insert(new EarlyReorderRecommendation($p.id));
end
```

Ako sistem detektuje da prodaja raste kroz tri uzastopna perioda, proizvod dobija oznaku rastućeg trenda. Ako se uz rastući trend zalihe približavaju minimalnom pragu, sistem generiše preporuku za ranije naručivanje. Time se sprečava situacija da popularan proizvod ostane bez zaliha.

```
rule "Smanjenje prodaje u tri uzastopna perioda"
when
    $p : Product()
    $s1 : SalesPeriod(productId == $p.id, period == 1, quantitySold > 0)
    $s2 : SalesPeriod(productId == $p.id, period == 2, quantitySold < $s1.quantitySold)
    $s3 : SalesPeriod(productId == $p.id, period == 3, quantitySold < $s2.quantitySold)
then
    insert(new DecreasingSalesTrend($p.id));
end
```

```
rule "Prevelike zalihe uz slabu prodaju"
when
    $p : Product()
    $trend : DecreasingSalesTrend(productId == $p.id)
    $stock : Stock(productId == $p.id, quantity > maxStockThreshold)
then
    insert(new ReduceOrderRecommendation($p.id));
end
```

Ako sistem detektuje da prodaja opada kroz tri uzastopna perioda, proizvod dobija oznaku opadajućeg trenda. Ako se uz opadajući trend zalihe povećavaju ili ostaju visoke, sistem generiše preporuku za smanjenje količine naredne narudžbine. Time se sprečava gomilanje zaliha i smanjuje rizik od nepotrebnih troškova skladištenja.

CEP pravila analiziraju događaje u vremenskim prozorima od 7 ili 30 dana kako bi sistem mogao da detektuje trendove u prodaji i promene u stanju zaliha.

Napomena: konkurentske cene se unose kao trenutni opseg i ne prate se kroz vreme.

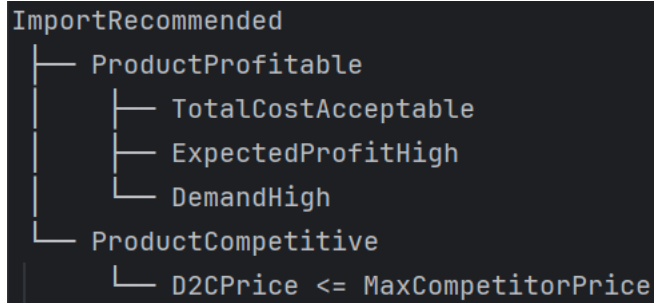
Backward chaining

Backward chaining se koristi kada korisnik želi objašnjenje ili proveru određene odluke. Sistem tada polazi od cilja, na primer Proizvod je isplativ za uvoz, i unazad proverava koji uslovi moraju biti ispunjeni da bi taj cilj bio tačan.

Primer cilja:

```
Goal: ImportRecommended(productId)
```

Da bi cilj bio tačan, sistem proverava:



Sistem kod backward chaining-a polazi od cilja ImportRecommended i proverava koje podciljeve mora da dokaže. Da bi proizvod bio preporučen za uvoz, mora biti profitabilan i konkurentan. Profitabilnost zavisi od prihvatljivog ukupnog troška, očekivanog profita i potražnje, dok konkurentnost zavisi od toga da li se D2C cena uklapa u konkurentski opseg.

Primer objašnjenja:

```
Pitanje: Zašto proizvod nije preporučen za uvoz?

Sistem proverava cilj: ImportRecommended.
Cilj nije ispunjen jer proizvod nije konkurentan.
Proizvod nije konkurentan jer je preporučena D2C cena 2300 RSD,
a maksimalna konkurentska cena je 1900 RSD.
Dodatno, potražnja je ocenjena kao niska, pa je očekivani profit mali.
```

Prednost backward chaining mehanizma je u transparentnosti sistema. Korisnik ne dobija samo konačnu odluku, već i lanac razloga koji je doveo do te odluke.

Template mehanizam

Template mehanizam je šablon koji generiše slična pravila za različite tipove proizvoda.

Na primer, kategorije:

```
luxury
mass
professional
trend
standard
```

Template može da definiše različite pragove:

Tip proizvoda	Minimalna zaliha	Prag za upozorenje	Strategija
mass	20	10	standardna
luxury	8	4	oprezna
professional	15	7	stabilna
trend	30	15	agresivna

Primer template pravila:

```
template header
productType
minStock
warningThreshold
orderMultiplier

rule "Stock warning for @{productType} products"
when
    $p : Product(type == "@{productType}")
    $s : Stock(productId == $p.id, quantity < @{warningThreshold})
then
    insert(new StockWarning($p.id, "@{productType}"));
end

rule "Order recommendation for @{productType} products"
when
    $p : Product(type == "@{productType}")
    $s : Stock(productId == $p.id, quantity < @{minStock})
then
    insert(new OrderRecommendation($p.id, @{orderMultiplier}));
end
```

Template podaci:

```
mass, 20, 10, 1.0
luxury, 8, 4, 0.7
professional, 15, 7, 1.2
trend, 30, 15, 1.5
```

Template mehanizam omogućava da se ista logika primeni na više grupa proizvoda, ali sa različitim parametrima. Na primer, trend proizvodi imaju viši minimalni prag zaliha i agresivniju strategiju naručivanja, dok luxury proizvodi imaju oprezniju strategiju jer su skuplji i obično imaju manji obim prodaje.

Ukratko:

Forward chaining donosi osnovnu odluku: cena, isplativost, uvoz zalihe.

Accumulate funkcija računa prosečnu prodaju i preostale dane zaliha.

CEP prati promene kroz vreme i detektuje trendove.

Templates prilagođava pragove po tipu proizvoda.

Backward chaining objašnjava zašto je sistem doneo neku odluku.

Primer rezonovanja

1. Unosi se proizvod i svi relevantni podaci (troškovi, potražnja, konkurentski opseg cena).
2. Sistem računa ukupni trošak proizvoda:
 $\text{ukupni trošak} = \text{nabavna cena} + \text{transport} + \text{carina}.$
3. Na osnovu ukupnog troška, sistem računa preporučene cene:
 - o $\text{D2C cena} = \text{ukupni trošak} + 45\%$
 - o $\text{B2B cena} = \text{ukupni trošak} + 30\%$
4. Sistem poredi D2C cenu sa konkurentskim opsegom:
 - o ako je cena iznad maksimalne konkurentske $\rightarrow$ proizvod nije konkurentan
5. Na osnovu konkurentnosti i potražnje, sistem procenjuje isplativost:
 - o ako je proizvod konkurentan i potražnja visoka $\rightarrow$ proizvod je isplativ
6. Ako je proizvod isplativ $\rightarrow$ sistem preporučuje uvoz.
7. Sistem analizira istoriju prodaje (accumulate + CEP):
 - o ako prodaja raste kroz više perioda $\rightarrow$ detektuje se rastući trend
8. Na osnovu trenda i stanja zaliha:
 - o ako prodaja raste i zalihe su male $\rightarrow$ povećava se količina narudžbine
 - o ako zalihe padnu ispod praga $\rightarrow$ generiše se upozorenje